16-Bit Pipelined RISC Processor using Verilog HDL

Project Report

1. Introduction

The 16-Bit Pipelined RISC Processor is a hardware design project implemented using Verilog HDL. The processor follows a Reduced Instruction Set Computing (RISC) architecture and utilizes pipelining techniques to improve instruction throughput and overall system performance.

The processor is divided into five execution stages:

- Instruction Fetch (IF)
- Instruction Decode (ID)
- Execute (EX)
- Memory Access (MEM)
- Write Back (WB)

To improve execution efficiency, the design incorporates hazard handling mechanisms including Data Forwarding and Hazard Detection Units.

---

2. Objectives

The main objectives of this project are:

- Design a 16-bit RISC processor using Verilog HDL
- Implement a 5-stage pipeline architecture
- Support arithmetic and logical instructions
- Implement memory access instructions
- Reduce pipeline stalls using forwarding
- Detect and handle data hazards
- Verify processor operation through simulation
- Develop a modular and scalable CPU architecture

---

3. Features of the Processor

Main Features

1. 16-bit RISC Architecture

2. Five-Stage Pipeline
3. Arithmetic Logic Unit (ALU)
4. Register File
5. Control Unit
6. Instruction Memory
7. Data Memory
8. Pipeline Registers
9. Data Forwarding Unit
10. Hazard Detection Unit
11. Write Back Mechanism
12. Verilog HDL Implementation

---

4. Overall System Architecture

The processor consists of the following major modules:

Program Counter

↓

Instruction Memory

↓

Instruction Decode / Control Unit

↓

Register File

↓

ALU

↓

Data Memory

↓

Write Back

Additional Supporting Modules:

- Pipeline Registers
- Forwarding Unit
- Hazard Detection Unit

---

5. Pipeline Architecture

The processor follows a classic five-stage pipeline.

Stage 1: Instruction Fetch (IF)

Functions:

- Fetch instruction from instruction memory
- Update Program Counter

Stage 2: Instruction Decode (ID)

Functions:

- Decode opcode
- Generate control signals
- Read source registers

Stage 3: Execute (EX)

Functions:

- Perform ALU operations
- Calculate memory addresses

Stage 4: Memory Access (MEM)

Functions:

- Read data memory
- Write data memory

Stage 5: Write Back (WB)

Functions:

- Write results back into register file

---

6. Hazard Handling Mechanisms

6.1 Data Forwarding Unit

Purpose:

Reduces data hazards by forwarding results directly to dependent instructions.

Functions:

- EX/MEM forwarding
- MEM/WB forwarding
- Eliminates unnecessary stalls

Benefits:

- Improved throughput
- Reduced execution delays

6.2 Hazard Detection Unit

Purpose:

Detects situations where forwarding alone is insufficient.

Functions:

- Load-use hazard detection
- Pipeline stalling
- Bubble insertion

Benefits:

- Correct instruction execution
- Improved reliability

---

7. Module Description

7.1 Program Counter

Purpose:

Maintains the address of the next instruction.

Functions:

- Sequential instruction execution
- PC increment

7.2 Instruction Memory

Purpose:

Stores processor instructions.

Functions:

- Instruction fetching
- Program execution support

7.3 Control Unit

Purpose:

Generates control signals for all pipeline stages.

Functions:

- Opcode decoding
- ALU control generation
- Pipeline control

7.4 Register File

Purpose:

Stores processor registers.

Functions:

- Read source operands
- Write destination results

7.5 Arithmetic Logic Unit (ALU)

Purpose:

Performs arithmetic and logical operations.

Supported Operations:

- ADD
- SUB
- AND
- OR
- XOR
- NOT
- SHL
- SHR

7.6 Data Memory

Purpose:

Stores runtime data.

Functions:

- LOAD operations
- STORE operations

7.7 Pipeline Registers

Purpose:

Separate pipeline stages.

Types:

- IF/ID
- ID/EX
- EX/MEM
- MEM/WB

---

8. Instruction Set Architecture (ISA)

Opcode	Instruction	Operation
0000	ADD	RD = RS1 + RS2
0001	SUB	RD = RS1 - RS2
0010	AND	RD = RS1 & RS2
0011	OR	RD = RS1 | RS2
0100	XOR	RD = RS1 ^ RS2
0101	NOT	RD = ~RS1
0110	SHL	RD = RS1 << RS2
0111	SHR	RD = RS1 >> RS2
1000	LOAD	Memory Read
1001	STORE	Memory Write
1010	ADDI	Immediate Addition

---

9. Test Cases Implemented

Test Case	Description
TC1	ADD Instruction
TC2	SUB Instruction
TC3	AND Operation
TC4	OR Operation
TC5	XOR Operation
TC6	Shift Left
TC7	Shift Right
TC8	LOAD Instruction
TC9	STORE Instruction
TC10	Forwarding Verification
TC11	Hazard Detection Verification
TC12	Pipeline Execution Validation

---

10. Simulation Tools Used

Software:

- Icarus Verilog (iverilog)
- GTKWave
- Visual Studio Code

Compilation

iverilog -o processor_sim tb/tb_processor.v src/*.v

Run Simulation

vvp processor_sim

Open Waveforms

gtkwave dump.vcd

```
PS C:\Users\bhavi\OneDrive\Desktop\iverilog\16bit_pipelined_risc> vvp processor_sim
VCD info: dumpfile processor_wave.vcd opened for output.
  16-bit RISC Processor Testbench

[RESET released at t=36000]

Pipeline monitor (first 5 cycles after reset)
  t=46000 PC=1 stall=0 flush=0/0 fwdA=00 fwdB=00 | WB:R0<=0x0000 we=0
  t=56000 PC=2 stall=0 flush=0/0 fwdA=00 fwdB=00 | WB:R0<=0x0000 we=0
  t=66000 PC=3 stall=0 flush=0/0 fwdA=00 fwdB=00 | WB:R0<=0x0000 we=1
  t=76000 PC=4 stall=0 flush=0/0 fwdA=00 fwdB=00 | WB:R1<=0x000a we=1
  t=86000 PC=5 stall=0 flush=0/0 fwdA=00 fwdB=01 | WB:R2<=0x0005 we=1

Waiting for HALT
  [INFO] HALT reached after ~4 cycles at t=126000

TEST 1 & 2: LOAD Instructions
  [PASS] R1 = 0x000a (expected 0x000a)
  [PASS] R2 = 0x0005 (expected 0x0005)

 TEST 3: ADD Instruction
  [PASS] R3 = 0x000f (expected 0x000f)

 TEST 4: STORE Instruction
  [PASS] mem[30] = 0x000f (expected 0x000f)

 TEST 5: R0 Hardwired Zero
  [PASS] R0 = 0x0000 (expected 0x0000)

 TEST 6: HALT Signal
  [PASS] halt_out asserted correctly

TEST 7: Reset Clears Pipeline
  [PASS] All pipeline registers cleared on reset
  RESULTS: 7 passed, 0 failed
  ALL TESTS PASSED
tb/tb_processor.v:185: $finish called at 296000 (1ps)
PS C:\Users\bhavi\OneDrive\Desktop\iverilog\16bit_pipelined_risc>
```

---

11. Waveform Analysis

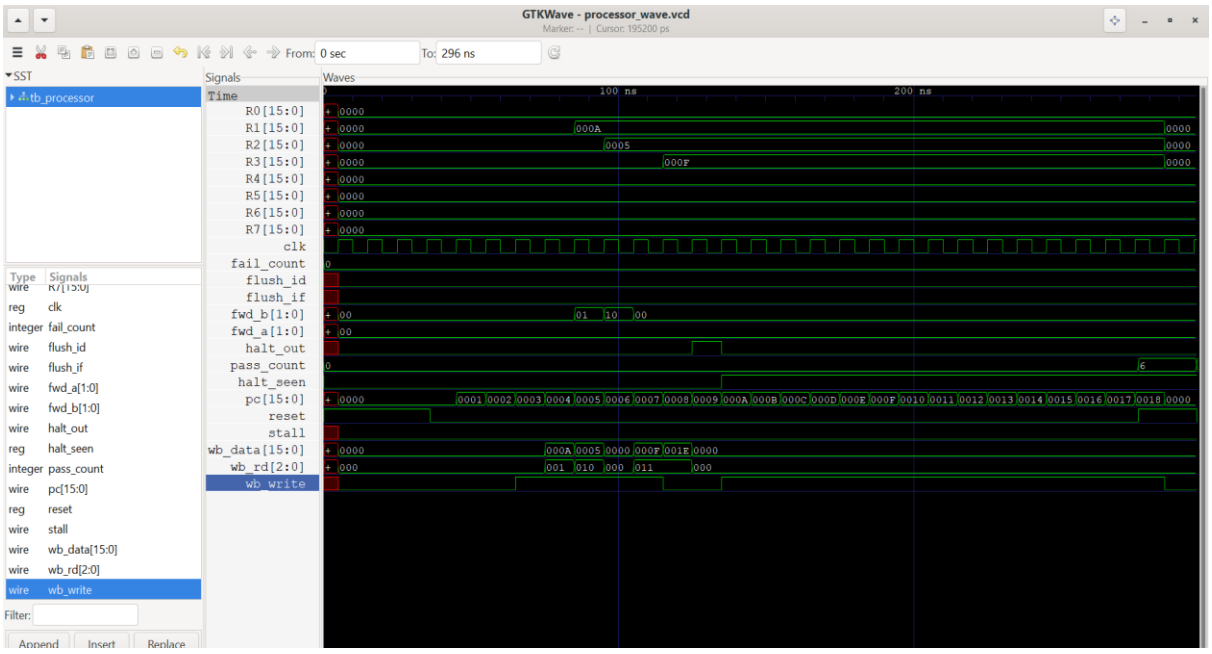
The waveform verifies:

- Program Counter operation
- Instruction Fetch
- Instruction Decode
- ALU execution
- Memory access
- Register write-back
- Pipeline progression
- Forwarding functionality

- Hazard detection functionality

Important Signals Observed:

- clk
- reset
- pc
- instruction
- alu_result
- mem_read
- mem_write
- reg_write
- forward_a
- forward_b
- stall



---

12. Advantages of the System

- Improved throughput through pipelining
- Reduced execution time
- Modular architecture
- Scalable design
- Efficient hazard handling

- Suitable for FPGA implementation
- Educational CPU design platform

---

13. Applications

- Computer Architecture Education
- FPGA Learning Projects
- Embedded Systems Research
- Processor Design Studies
- Digital System Verification
- Hardware Design Training

---

14. Future Enhancements

Future improvements may include:

- Branch prediction unit
- Cache memory support
- Interrupt handling
- Floating-point operations
- 32-bit architecture upgrade

---

15. Conclusion

The 16-Bit Pipelined RISC Processor successfully demonstrates the implementation of a complete pipelined CPU using Verilog HDL. The processor integrates instruction execution, memory access, forwarding logic, and hazard detection into a modular architecture.

The project provides practical understanding of:

- Computer Architecture
- Pipeline Design
- Hazard Management
- RTL Design
- Verilog HDL
- Digital System Verification